

Quiz

1. Write the asymptotic time complexity of the following code snippet. Show your works/reasoning. [4]

```
for(i = n; i >= 0; i = i - 4){  
    for (j = 1; j < n; j = j * 3){  
        for (k = 1; k <= 20; k++){  
            m = m + 1  
        }  
    }  
}
```

$\rightarrow n, n-4, n-8, \dots, 8, 4, 1$ etc.

$\rightarrow 1, 3, 9, 27, \dots, n$

$\rightarrow$ no matter what value of n ,
will run 21 times so constant $O(1)$

$$\frac{n}{4}$$

$$n = 3^k \quad k = \log_3 n$$

$$\rightarrow O\left(\frac{n}{4} \times \log_3 n \times 1\right) = O(n \log_3 n)$$

1. Write the asymptotic time complexity of the following code snippet. Show your works/reasoning. [4]

```
for(i = 0; i <= n; i = i + 5){  
    for (j = n; j >= 1; j = j / 4){  
        for (k = 30; k >= 1; k - -){  
            m = m + 1  
        }  
    }  
}
```

$\rightarrow n/5$

$\rightarrow \log_4 n$

$\rightarrow$ constant

$$\rightarrow O(n \log_4 n)$$

2. Suppose you are given an array, a of length n and an index, i of the array. The elements of the array are arranged in such a way that the values from index 0 to i are sorted in ascending order and the values from index $i+1$ to $n-1$ are sorted in descending order. Moreover it is ensured that all the values of the second part of the array ($i+1 \dots n-1$) are greater than the values in the first part of the array. See the given array for better understanding.

[1, 2, 3, 4, 9, 8, 7, 6, 5], $i = 3$

Now propose an algorithm to search a number, num in the array. Your algorithm should take less than linear time. Express your algorithm in code/pseudocode/flowchart (any method is acceptable) [5]

$\rightarrow$ This is a binary search problem with a twist.

$0 \rightarrow (i) \rightarrow$ ascending
 $i+1 \rightarrow (n-1) \rightarrow$ descending

So, either i th value (4) or the $n-1$ th value (5) divides the array in two halves which are sorted by themselves.

So, we need to compare our number n (the one we want to search) with either $a[i]$ or $a[n-1]$

if, $n > a[i]$ it must be in the second half.
on $n > a[n-1]$

say, $n = 7 \rightarrow 7 > 4$ on $7 > 5$

again,

if $n < a[i]$ it must be in the first half
on $n < a[n-1]$

say $n = 2$ $2 < 4$ on $2 < 5$

So, first we find which sorted part the number belongs by the mentioned comparison.

If the number is in the ascending part
do normal binary search.

if it is in descending part,

modify the search.

if $n < mid \rightarrow$ go to right (normally we go left)
 $l = mid + 1$

if $n > mid \rightarrow$ go to left (normally we go right)
 $r = mid - 1$

That's it.

2. Suppose you are given an **array, a** of length **n** and an **index, i** of the array. The elements of the array are arranged in such a way that the **values from index 0 to i** are sorted in **descending order** and the values from **index i+1 to n-1** are sorted in **ascending order**. Moreover it is ensured that all the values of the second part of the array($i+1 \dots n-1$) are **greater** than the values in the first part of the array. See the given array for better understanding.

[4, 3, 2, 1, 5, 6, 7, 8, 9], $i = 3$

Now propose an algorithm to **search a number, num** in the array. Your algorithm **should take less than linear time**. Express your algorithm in code/pseudocode/flowchart (any method is acceptable)

[5]

→ similar problem → but the partition is done by $a[0]$ on $a[i+1]$

if $num < a[0]$
on $num < a[i+1]$

→ search in the first part
(reverse binary search)

if $num > a[0]$
or $num > a[i+1]$

→ search in the second part
(normal binary search)

1 a. Consider the following functions.

03

CO2

$$f_1(n) = \log(n!)$$

$$f_2(n) = n^2 \log_n(n^n)$$

$$f_3(n) = n^4 + 10n^2$$

$$f_4(n) = 17 \cdot 13^n$$

$$f_5(n) = n \log n$$

$$f_6(n) = n * \sqrt[5]{n^2}$$

Write a correct asymptotic upper bound for each of the above and sort the functions in ascending order of their growth rate.

$$\begin{aligned} \textcircled{1} \log(n!) &= \log(n \times (n-1) \times (n-2) \cdots \times (1)) \\ &= \log n + \log(n-1) + \log(n-2) + \cdots + \log(1) \\ &\quad \downarrow \quad \downarrow \quad \downarrow \quad \vdots \\ &\quad o(\log n) \quad o(\log n) \quad o(\log n) \quad o(\log n) \\ &\quad \text{(upper bound, so, take anything greater)} \\ &= O(n \log n) \end{aligned}$$

$$\textcircled{2} n^2 \log_n(n^n) \longrightarrow n^2 n \log_n n \longrightarrow n^3 \quad O(n^3)$$

$$\textcircled{3} n^4 + 10n^2 \longrightarrow O(n^4)$$

$$\textcircled{4} 17 \cdot 13^n \longrightarrow O(18^n)$$

$$\textcircled{5} n \log n \longrightarrow O(n \log n)$$

$$\begin{aligned} \textcircled{6} n * \sqrt[5]{n^2} &= n \times n^{2/5} = n^{1 + \frac{2}{5}} = n^{\frac{7}{5}} = n^{3.5} \\ &= O(n^4) \end{aligned}$$

```

1. for i in range (1,n)
2.     j= 1
3.     while j < i*i
4.         j= j-1

```

always positive

j decreases
so, loop goes on forever

$O(\infty)$

Express the following (either one of the two) running time $T(n)$ with an asymptotic bound.

$$T(n) = 8T\left(\frac{n}{4}\right) + n\sqrt{n} \quad \text{Or} \quad T(n) = 2T(n-2) + 1$$

Any method is acceptable as long as you show calculations.

① $T(n) = 8T(n/4) + n\sqrt{n}$

$a = 8 \quad b = 4$

$f(n) = n^{1+\frac{1}{2}} = n^{\frac{3}{2}}$

master method.

② $T(n) = 2T(n-2) + 1$

$= 2(2T(n-4) + 1) + 1$

$= 4T(n-4) + 2 + 1$

$= 4(2T(n-6) + 1) + 2 + 1$

$= 8T(n-6) + (4 + 2 + 1)$

sum of even number

$2^3 \quad \vdots \quad (n - 2 \times k)$

find pattern

- b. Consider an array containing N unique values where for some index i , the values are in increasing order from index 0 to $(i-1)$, and then again from i to $(N-1)$. Moreover, it is guaranteed that all the values from index 0 to $(i-1)$ are greater than all the values from i to $(N-1)$.

An example array is given below.

index	0	1	2	3	4	5	6	7
value	9	12	15	2	4	5	7	8

Here $i=3$, it means the values are in increasing order from index 0 to 2 , and then again from 3 to 7 . Also, all values from index 0 to 2 are greater than all values from 3 to 7 (it is guaranteed, no checking required).

Given such an array, propose an algorithm to find the index i .

- CO3 i) Write your algorithm with a code/pseudocode/flowchart/step-by-step instructions.
CO2 ii) Write the time complexity of your algorithm.

04
02

Similar to quiz question.

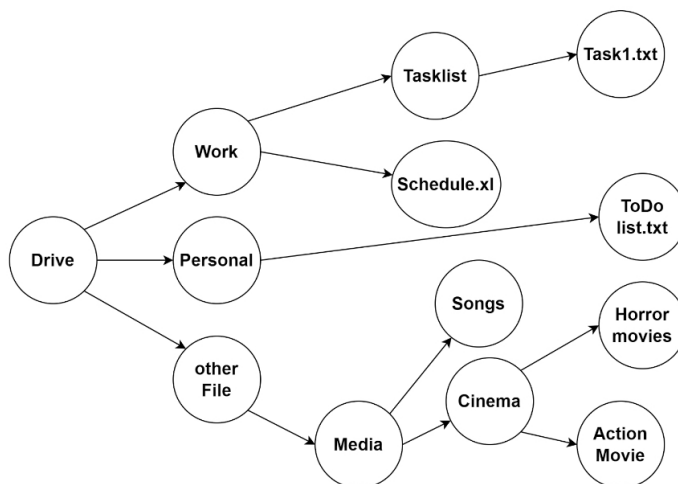
check the value in $arr[n-1]$.

if $num < arr[n-1]$ it is in $(i, n-1)$
do binary search.

else in the first part.

Since all the values in the second part are smaller than all the values in the first part.

- 4 CO1 You are a computer science student and you are given a file structure encoded in a graph. You want to make a navigator system where if a person wants to find a file they can enter the name of the file and your program will return the exact location of the file. For example, consider the following graph:



pretty straightforward
do bfs or dfs with
parent tracker.
but since each
destination has only
one path, dfs
is preferable.

If the user wants to find task1.txt; your program will give the following output:

Drive->Work->Tasklist->task1.txt

If the user wants to find Horror Movies; your program will give the following output:

Drive->other file->Media->Cinema->Horror Movies

Now answer the following questions:

- Give the adjacency list representation of this graph. You can use either the whole name or a shorter version of each node.
- Write the name of your preferred algorithm to solve the above mentioned problem (a navigator system). Explain your reasoning in brief.
- Show a simulation of your presented solution with proper use of data structure and other necessary details to give the desired answers as the sample input shows.

03

02

05